Intelligence Artificielle

Logique, SAT et Programmation par Contraintes

Jean-Guy Mailly (FI), Leila Moudjari (FA)

Master 1 MIAGE – 2025-2026

TD: Audren Boulic-Bouadjio, Leila Moudjari, Paul Saves

Introduction

Espaces d'états vs Espaces de variables

Modélisation par espaces d'états

- Avantages : modélisation plus naturelle et possibilité de calculer des séquences d'actions
- Inconvénients : il faut considérer un nombre d'états possible très grand (exponentiel par rapport aux nombre de « composants » dans les états)

Modélisation des états avec un ensemble de variables

- Inconvénient : modélisation potentiellement plus complexe
- Avantages : pas besoin d'énumérer les états, possibilité d'utiliser des algorithmes très efficaces (solveurs) pour résoudre des gros problèmes rapidement

Espaces d'états vs Espaces de variables

Modélisation par espaces d'états

- Avantages : modélisation plus naturelle et possibilité de calculer des séquences d'actions
- Inconvénients : il faut considérer un nombre d'états possible très grand (exponentiel par rapport aux nombre de « composants » dans les états)

Modélisation des états avec un ensemble de variables

- Inconvénient : modélisation potentiellement plus complexe
- Avantages : pas besoin d'énumérer les états, possibilité d'utiliser des algorithmes très efficaces (solveurs) pour résoudre des gros problèmes rapidement

On ne considère plus les chemins entre l'état initial et l'état final, mais uniquement les valeurs des variables dans l'état but

Exemple : Le Sudoku

	2		5		1		9	
8			2		3			6
	3			6			7	
		1				6		
5	4						1	9
		2				7		
	9			3			8	
2			8		4			7
	1		9		7		6	

Résolution avec la recherche arborescente

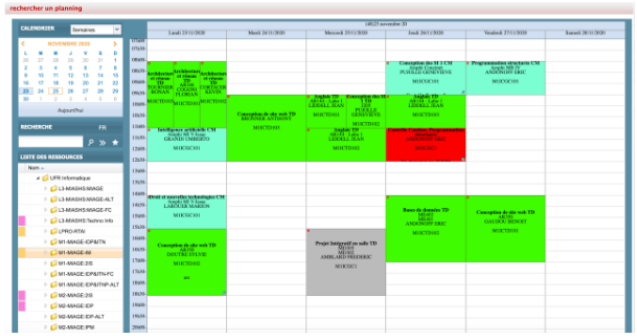
- Action : placer un nombre dans une case libre
- États buts : la grille est pleine, et il n'y a pas deux fois la même valeur dans une ligne, une colonne, ou un secteur 3×3
- Facteur de branchement : très grand ! (81 si la grille de départ est vide)

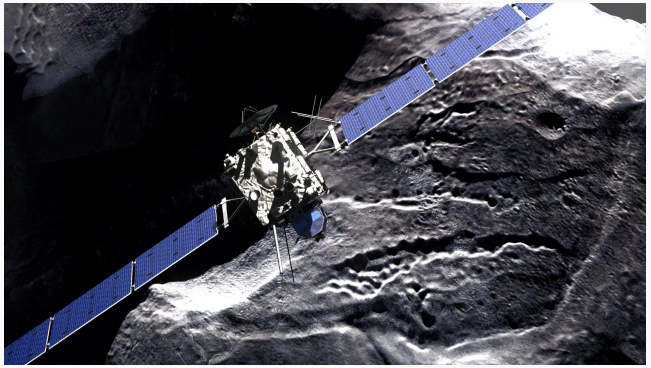
Résolution avec la programmation par contraintes

- 81 variables (une par case) et des contraintes pour décrire les règles du Sudoku
- un solveur de contraintes fournit une solution (ou la preuve qu'il n'en existe pas)

Applications

- Emplois du temps
- Allocation de tâches ou de ressources (entre des travailleurs, des citoyens, des processeurs,. . .)
- Ordonnancement/planifications de tâches (production d'avion chez Airbus, observations de la sonde spatiale Rosetta)





https://www.a4cp.org/node/1058

Application : Rosetta

- Mission : recueillir des données sur la composition du noyau de la comète Tchouri (https://fr.wikipedia.org/wiki/Rosetta_(sonde_spatiale))
- La sonde dispose de peu d'énergie et d'un temps limité pour décider quelles expériences et mesures effectuer en fonction des conditions observées sur place
- Les scientifiques souhaitent réaliser un maximum d'expériences en un minimum de temps, étant donné le coût très élevé de la mission
- Plusieurs pays ont financé la mission, mais pas tous avec le même montant : comment répartir le temps d'expérimentation entre eux de manière équitable ?

Gestion d'une cargaison

Nombreuses contraintes pour la sécurité et l'efficacité du transport de marchandises

- Répartition équilibrée du poids
- L'ordre dans lequel le cargo visite les ports est important : on ne veut pas être obligé de décharger tout le cargo pour accéder aux conteneurs du premier port !
- De manière générale : minimisation du temps de chargement/déchargement

Logique et SAT

Logique Propositionnelle : les bases

- Cadre formel simple pour exprimer des contraintes
- Manipulation de *propositions* : énoncés qui peuvent être *V*rai ou *F*aux
  - *Il pleut*, *Il fait du soleil*, *Nous sommes à Toulouse*, *Nous sommes à Paris*
  - *Il y a un* 1 *dans la première case de la première ligne*
- Combinaison de propositions (*ET*, *OU*, *NON*, *SI X ALORS Y*, ...)
  - *Nous sommes à Toulouse OU nous sommes à Paris*, *SI nous sommes à Toulouse ALORS il y a du soleil*
  - *SI il y a un* 1 *dans la première case de la première ligne, ALORS il n'y a pas de* 1 *ailleurs sur la première ligne*
- Ce qu'on met généralement dans un `if` ou un `while`, c'est de la logique propositionnelle !

Syntaxe de la logique propositionnelle

- On définit un vocabulaire $\mathcal{V} = \{x_1, \ldots, x_n\}$: ensemble de variables qui peuvent recevoir une valeur *V*rai (parfois noté 1, ou $\top$) ou *F*aux (0, $\bot$)
- Une formule propositionnelle ϕ est
  - un atome (juste une variable) : $\phi = x$
  - une négation (*NON*) : $\phi = \neg\phi_1$
  - une conjonction (*ET*) : $\phi = \phi_1 \wedge \phi_2$
  - une disjonction (*OU*) : $\phi = \phi_1 \vee \phi_2$
  - une implication (*SI* ... *ALORS*) : $\phi = \phi_1 \rightarrow \phi_2$
  - une équivalence (*SI ET SEULEMENT SI*) : $\phi = \phi_1 \leftrightarrow \phi_2$
  - (et on peut en définir d'autres...)
- *SI nous sommes à Toulouse ALORS il y a du soleil* devient *toulouse* $\rightarrow$ *soleil*
- Pour le Sudoku : $x_i^{j,k}$ représente « il y a la valeur i dans la case j de la ligne k »
  - *SI il y a un* 1 *dans la première case de la première ligne, ALORS il n'y a pas de* 1 *ailleurs sur la première ligne* devient $x_1^{1,1} \rightarrow (\neg x_1^{2,1} \wedge \cdots \wedge \neg x_1^{9,1})$

Sémantique de la logique propositionnelle (1/2)

- Sémantique : association d'une valeur 0 ou 1 à chaque variable (interprétation)
- Des règles permettent de connaître la valeur d'une formule à partir de l'interprétation des variables
- Table de vérité : représentation de la valeur d'une formule pour chaque interprétation possible
- Par exemple :
  - $\neg\phi$ est vrai quand ϕ est faux
  - $\phi_1 \wedge \phi_2$ est vrai quand ϕ_1 et ϕ_2 sont vrais tous les deux

x	$\neg x$
0	1
1	0

x	y	$x \wedge y$
0	0	0
0	1	0
1	0	0
1	1	1

- S'il y a n variables, il y a 2^n lignes dans la table !

Sémantique de la logique propositionnelle (2/2)

- $\phi_1 \vee \phi_2$ est vrai quand au moins un parmi ϕ_1 et ϕ_2 est vrai
- $\phi_1 \rightarrow \phi_2$ est vrai quand ϕ_1 et ϕ_2 sont vrais, ou ϕ_1 est faux
- $\phi_1 \leftrightarrow \phi_2$ est vrai quand ϕ_1 et ϕ_2 ont la même valeur

x	y	$x \vee y$
0	0	0
0	1	1
1	0	1
1	1	1

x	y	$x \rightarrow y$
0	0	1
0	1	1
1	0	0
1	1	1

x	y	$x \leftrightarrow y$
0	0	1
0	1	0
1	0	0
1	1	1

- On peut combiner l'utilisation de ce principe pour savoir quand une formule plus complexe est vraie
- Les interprétations où la formule vaut 1 sont appelées les *modèles* de la formule

Exemple simple : l'implication revisitée

- $\phi = \neg x \vee y$

x	y	$\neg x \vee y$
0	0	1
0	1	1
1	0	0
1	1	1

Exemple simple : l'implication revisitée

- $\phi = \neg x \vee y$

x	y	$\neg x \vee y$
0	0	1
0	1	1
1	0	0
1	1	1

x	y	$x \to y$
0	0	1
0	1	1
1	0	0
1	1	1

- $\phi_1 \to \phi_2$ est équivalent à $\neg\phi_1 \vee \phi_2$!

Exemple simple : le “ou exclusif”

- $\phi = x \oplus y$ représente le « ou exclusif »
- Rappel : avec une disjonction (le « ou classique » $\vee$), la formule est vraie si *au moins* une sous-formule est vraie
- Avec $\oplus$, la formule est vraie si *exactement* une sous-formule est vraie

x	y	$x \oplus y$
0	0	0
0	1	1
1	0	1
1	1	0

Exemple simple : le “ou exclusif”

- $\phi = x \oplus y$ représente le « ou exclusif »
- Rappel : avec une disjonction (le « ou classique » $\vee$), la formule est vraie si *au moins* une sous-formule est vraie
- Avec $\oplus$, la formule est vraie si *exactement* une sous-formule est vraie

x	y	$x \oplus y$
0	0	0
0	1	1
1	0	1
1	1	0

x	y	$x \leftrightarrow y$
0	0	1
0	1	0
1	0	0
1	1	1

- $\phi_1 \oplus \phi_2$ est équivalent à $\neg(\phi_1 \leftrightarrow \phi_2)$!

Un exemple plus complexe

- $\phi = (x \vee \neg y) \rightarrow (x \wedge z)$
- On peut décomposer la formule en sous-formules plus simples

x	y	z	$\neg y$	$x \vee \neg y$	$x \wedge z$	ϕ
0	0	0				
0	0	1				
0	1	0				
0	1	1				
1	0	0				
1	0	1				
1	1	0				
1	1	1				

Un exemple plus complexe

- $\phi = (x \vee \neg y) \rightarrow (x \wedge z)$
- On peut décomposer la formule en sous-formules plus simples

x	y	z	$\neg y$	$x \vee \neg y$	$x \wedge z$	ϕ
0	0	0	1			
0	0	1	1			
0	1	0	0			
0	1	1	0			
1	0	0	1			
1	0	1	1			
1	1	0	0			
1	1	1	0			

- $\neg y$ est vrai quand y est faux

Un exemple plus complexe

- $\phi = (x \vee \neg y) \rightarrow (x \wedge z)$
- On peut décomposer la formule en sous-formules plus simples

x	y	z	$\neg y$	$x \vee \neg y$	$x \wedge z$	ϕ
0	0	0	1	1		
0	0	1	1	1		
0	1	0	0	0		
0	1	1	0	0		
1	0	0	1	1		
1	0	1	1	1		
1	1	0	0	1		
1	1	1	0	1		

- $\neg y$ est vrai quand y est faux
- $x \vee \neg y$ est vrai quand x est vrai ou $\neg y$ est vrai

Un exemple plus complexe

- $\phi = (x \vee \neg y) \rightarrow (x \wedge z)$
- On peut décomposer la formule en sous-formules plus simples

x	y	z	$\neg y$	$x \vee \neg y$	$x \wedge z$	ϕ
0	0	0	1	1	0	
0	0	1	1	1	0	
0	1	0	0	0	0	
0	1	1	0	0	0	
1	0	0	1	1	0	
1	0	1	1	1	1	
1	1	0	0	1	0	
1	1	1	0	1	1	

- $\neg y$ est vrai quand y est faux
- $x \vee \neg y$ est vrai quand x est vrai ou $\neg y$ est vrai
- $x \wedge z$ est vrai quand x est z sont vrais tous les deux

Un exemple plus complexe

- $\phi = (x \vee \neg y) \rightarrow (x \wedge z)$
- On peut décomposer la formule en sous-formules plus simples

x	y	z	$\neg y$	$x \vee \neg y$	$x \wedge z$	ϕ
0	0	0	1	1	0	0
0	0	1	1	1	0	0
0	1	0	0	0	0	1
0	1	1	0	0	0	1
1	0	0	1	1	0	0
1	0	1	1	1	1	1
1	1	0	0	1	0	0
1	1	1	0	1	1	1

- $\neg y$ est vrai quand y est faux
- $x \vee \neg y$ est vrai quand x est vrai ou $\neg y$ est vrai
- $x \wedge z$ est vrai quand x est z sont vrais tous les deux
- l'implication (ϕ) est vraie quand ses deux parties sont vraies, ou la partie gauche est fausse

Exercice

- Trouvez la table de vérité de $\phi = (\neg x \vee \neg y) \wedge (x \vee \neg z)$

x	y	z	$\neg x \vee \neg y$	$x \vee \neg z$	ϕ
0	0	0			
0	0	1			
0	1	0			
0	1	1			
1	0	0			
1	0	1			
1	1	0			
1	1	1			

Exercice

- Trouvez la table de vérité de $\phi = (\neg x \vee \neg y) \wedge (x \vee \neg z)$

x	y	z	$\neg x \vee \neg y$	$x \vee \neg z$	ϕ
0	0	0	1		
0	0	1	1		
0	1	0	1		
0	1	1	1		
1	0	0	1		
1	0	1	1		
1	1	0	0		
1	1	1	0		

- $\neg x \vee \neg y$ est vrai quand x ou y est faux (ou les deux)

Exercice

- Trouvez la table de vérité de $\phi = (\neg x \vee \neg y) \wedge (x \vee \neg z)$

x	y	z	$\neg x \vee \neg y$	$x \vee \neg z$	ϕ
0	0	0	1	1	
0	0	1	1	0	
0	1	0	1	1	
0	1	1	1	0	
1	0	0	1	1	
1	0	1	1	1	
1	1	0	0	1	
1	1	1	0	1	

- $\neg x \vee \neg y$ est vrai quand x ou y est faux (ou les deux)
- $x \vee \neg z$ est vrai quand x est vrai ou $\neg z$ est vrai (donc z est faux)

Exercice

- Trouvez la table de vérité de $\phi = (\neg x \vee \neg y) \wedge (x \vee \neg z)$

x	y	z	$\neg x \vee \neg y$	$x \vee \neg z$	ϕ
0	0	0	1	1	1
0	0	1	1	0	0
0	1	0	1	1	1
0	1	1	1	0	0
1	0	0	1	1	1
1	0	1	1	1	1
1	1	0	0	1	0
1	1	1	0	1	1

- $\neg x \vee \neg y$ est vrai quand x ou y est faux (ou les deux)
- $x \vee \neg z$ est vrai quand x est vrai ou $\neg z$ est vrai (donc z est faux)
- ϕ est vrai quand les deux sous-formules sont vraies

Un petit casse-tête : tigres et princesses

Inspiré de *Le livre qui rend fou* (Raymond Smullyan)

- Dans un royaume ancien, le roi met un prisonnier face à deux cellules, qui peuvent contenir soit un tigre, soit un princesse
- Le prisonnier doit entrer dans une cellule. Si elle contient un tigre, il se fait dévorer, mais si elle contient une princesse il est libéré et épouse la princesse
- Des affiches sur les portes donnent des indices pour choisir, et le roi lui précise qu'une affiche dit la vérité, et l'autre ment
  - Porte 1 : « Il y a une princesse dans cette cellule et un tigre dans l'autre »
  - Porte 2 : « Il y a une princesse dans une cellule et il y a un tigre dans une cellule »
- Quelle porte choisiriez vous ?

Tigres et princesses, modélisation

On peut modéliser de différentes façons

- t_1, t_2 : il y a un tigre dans la cellule 1/cellule 2
- p_1, p_2 : il y a une princesse dans la cellule 1/cellule 2

Tigres et princesses, modélisation

On peut modéliser de différentes façons

- t_1, t_2 : il y a un tigre dans la cellule 1/cellule 2
- p_1, p_2 : il y a une princesse dans la cellule 1/cellule 2
- Problème : ça complexifie la résolution du problème ($2^4 = 16$ lignes pour la table de vérité, et il faut représenter explicitement le fait qu'il n'y a pas un tigre et une princesse dans la même cellule)

Tigres et princesses, modélisation

On peut modéliser de différentes façons

- t_1, t_2 : il y a un tigre dans la cellule 1/cellule 2
- p_1, p_2 : il y a une princesse dans la cellule 1/cellule 2
- Problème : ça complexifie la résolution du problème ($2^4 = 16$ lignes pour la table de vérité, et il faut représenter explicitement le fait qu'il n'y a pas un tigre et une princesse dans la même cellule)
- On n'a besoin que des variables t_1 et t_2 : pour dire qu'il y a une princesse dans une cellule, on peut utiliser $\neg t_1$ et $\neg t_2$

Tigres et princesses, modélisation

On peut modéliser de différentes façons

- t_1, t_2 : il y a un tigre dans la cellule 1/cellule 2
- p_1, p_2 : il y a une princesse dans la cellule 1/cellule 2
- Problème : ça complexifie la résolution du problème ($2^4 = 16$ lignes pour la table de vérité, et il faut représenter explicitement le fait qu'il n'y a pas un tigre et une princesse dans la même cellule)
- On n'a besoin que des variables t_1 et t_2 : pour dire qu'il y a une princesse dans une cellule, on peut utiliser $\neg t_1$ et $\neg t_2$
- Porte 1 : « Il y a une princesse dans cette cellule et un tigre dans l'autre », $\phi_1 = \neg t_1 \wedge t_2$

Tigres et princesses, modélisation

On peut modéliser de différentes façons

- t_1, t_2 : il y a un tigre dans la cellule 1/cellule 2
- p_1, p_2 : il y a une princesse dans la cellule 1/cellule 2
- Problème : ça complexifie la résolution du problème ($2^4 = 16$ lignes pour la table de vérité, et il faut représenter explicitement le fait qu'il n'y a pas un tigre et une princesse dans la même cellule)
- On n'a besoin que des variables t_1 et t_2 : pour dire qu'il y a une princesse dans une cellule, on peut utiliser $\neg t_1$ et $\neg t_2$
- Porte 1 : « Il y a une princesse dans cette cellule et un tigre dans l'autre », $\phi_1 = \neg t_1 \wedge t_2$
- Porte 2 : « Il y a une princesse dans une cellule et il y a un tigre dans une cellule », $\phi_2 = (\neg t_1 \vee \neg t_2) \wedge (t_1 \vee t_2)$

Tigres et princesses, modélisation

On peut modéliser de différentes façons

- t_1, t_2 : il y a un tigre dans la cellule 1/cellule 2
- p_1, p_2 : il y a une princesse dans la cellule 1/cellule 2
- Problème : ça complexifie la résolution du problème ($2^4 = 16$ lignes pour la table de vérité, et il faut représenter explicitement le fait qu'il n'y a pas un tigre et une princesse dans la même cellule)
- On n'a besoin que des variables t_1 et t_2 : pour dire qu'il y a une princesse dans une cellule, on peut utiliser $\neg t_1$ et $\neg t_2$
- Porte 1 : « Il y a une princesse dans cette cellule et un tigre dans l'autre », $\phi_1 = \neg t_1 \wedge t_2$
- Porte 2 : « Il y a une princesse dans une cellule et il y a un tigre dans une cellule », $\phi_2 = (\neg t_1 \vee \neg t_2) \wedge (t_1 \vee t_2)$
- Une affiche dit la vérité, et l'autre ment, $\phi = \phi_1 \leftrightarrow \neg\phi_2$

Tigres et princesses, résolution

- $\phi_1 = \neg t_1 \wedge t_2$, $\phi_2 = (\neg t_1 \vee \neg t_2) \wedge (t_1 \vee t_2)$, $\phi = \phi_1 \leftrightarrow \neg\phi_2$

t_1	t_2	$\phi_1 = \neg t_1 \wedge t_2$	$\neg t_1 \vee \neg t_2$	$t_1 \vee t_2$	ϕ_2	ϕ
0	0	0	1	0	0	0
0	1	1	1	1	1	0
1	0	0	1	1	1	1
1	1	0	0	1	0	0

Tigres et princesses, résolution

- $\phi_1 = \neg t_1 \wedge t_2$, $\phi_2 = (\neg t_1 \vee \neg t_2) \wedge (t_1 \vee t_2)$, $\phi = \phi_1 \leftrightarrow \neg\phi_2$

t_1	t_2	$\phi_1 = \neg t_1 \wedge t_2$	$\neg t_1 \vee \neg t_2$	$t_1 \vee t_2$	ϕ_2	ϕ
0	0	0	1	0	0	0
0	1	1	1	1	1	0
1	0	0	1	1	1	1
1	1	0	0	1	0	0

- Le seul modèle de ϕ : $t_1 = 1$ et $t_2 = 0$, donc il y a un tigre dans la cellule 1 et une princesse dans la cellule 2

Logique et Sudoku

Essayez (après le cours) de trouver une représentation logique des règles du Sudoku

- De quelles variables propositionnelles avez vous besoin ?
- Quelles règles logiques permettent de résoudre une grille de Sudoku ?

Forme Normale Conjonctive

- Les algorithmes pour raisonner avec des formules logiques utilisent la forme normale conjonctive (CNF, pour *Conjunctive Normal Form*)
- Toute formule propositionnelle peut-être transformée en une CNF équivalente (avec les mêmes modèles)

Forme Normale Conjonctive

- Les algorithmes pour raisonner avec des formules logiques utilisent la forme normale conjonctive (CNF, pour *Conjunctive Normal Form*)
- Toute formule propositionnelle peut-être transformée en une CNF équivalente (avec les mêmes modèles)
- Littéral : soit une variable seule, soit la négation d'une variable (x est un littéral, $\neg y$ est un littéral, mais $x \vee \neg y$ n'en est pas un !)

Forme Normale Conjonctive

- Les algorithmes pour raisonner avec des formules logiques utilisent la forme normale conjonctive (CNF, pour *Conjunctive Normal Form*)
- Toute formule propositionnelle peut-être transformée en une CNF équivalente (avec les mêmes modèles)
- Littéral : soit une variable seule, soit la négation d'une variable (x est un littéral, $\neg y$ est un littéral, mais $x \vee \neg y$ n'en est pas un !)
- Clause : disjonction de littéraux (au moins 1, ou plusieurs littéraux, par exemple $\neg y$ est une clause, $x \vee \neg y$ est une clause, mais $x \wedge \neg y$ n'en est pas une)

Forme Normale Conjonctive

- Les algorithmes pour raisonner avec des formules logiques utilisent la forme normale conjonctive (CNF, pour *Conjunctive Normal Form*)
- Toute formule propositionnelle peut-être transformée en une CNF équivalente (avec les mêmes modèles)
- Littéral : soit une variable seule, soit la négation d'une variable (x est un littéral, $\neg y$ est un littéral, mais $x \vee \neg y$ n'en est pas un !)
- Clause : disjonction de littéraux (au moins 1, ou plusieurs littéraux, par exemple $\neg y$ est une clause, $x \vee \neg y$ est une clause, mais $x \wedge \neg y$ n'en est pas une)
- Formule CNF : conjonction de clauses (au moins 1, ou plusieurs clauses)

Forme Normale Conjonctive

- Les algorithmes pour raisonner avec des formules logiques utilisent la forme normale conjonctive (CNF, pour *Conjunctive Normal Form*)
- Toute formule propositionnelle peut-être transformée en une CNF équivalente (avec les mêmes modèles)
- Littéral : soit une variable seule, soit la négation d'une variable (x est un littéral, $\neg y$ est un littéral, mais $x \vee \neg y$ n'en est pas un !)
- Clause : disjonction de littéraux (au moins 1, ou plusieurs littéraux, par exemple $\neg y$ est une clause, $x \vee \neg y$ est une clause, mais $x \wedge \neg y$ n'en est pas une)
- Formule CNF : conjonction de clauses (au moins 1, ou plusieurs clauses)
- Une interprétation satisfait une clause si elle satisfait au moins un littéral dans la clause (car c'est une disjonction)

Forme Normale Conjonctive

- Les algorithmes pour raisonner avec des formules logiques utilisent la forme normale conjonctive (CNF, pour *Conjunctive Normal Form*)
- Toute formule propositionnelle peut-être transformée en une CNF équivalente (avec les mêmes modèles)
- Littéral : soit une variable seule, soit la négation d'une variable (x est un littéral, $\neg y$ est un littéral, mais $x \vee \neg y$ n'en est pas un !)
- Clause : disjonction de littéraux (au moins 1, ou plusieurs littéraux, par exemple $\neg y$ est une clause, $x \vee \neg y$ est une clause, mais $x \wedge \neg y$ n'en est pas une)
- Formule CNF : conjonction de clauses (au moins 1, ou plusieurs clauses)
- Une interprétation satisfait une clause si elle satisfait au moins un littéral dans la clause (car c'est une disjonction)
- Une interprétation satisfait une CNF si elle satisfait toutes les clauses (car c'est une conjonction)

Forme Normale Conjonctive

- Les algorithmes pour raisonner avec des formules logiques utilisent la forme normale conjonctive (CNF, pour *Conjunctive Normal Form*)
- Toute formule propositionnelle peut-être transformée en une CNF équivalente (avec les mêmes modèles)
- Littéral : soit une variable seule, soit la négation d'une variable (x est un littéral, $\neg y$ est un littéral, mais $x \vee \neg y$ n'en est pas un !)
- Clause : disjonction de littéraux (au moins 1, ou plusieurs littéraux, par exemple $\neg y$ est une clause, $x \vee \neg y$ est une clause, mais $x \wedge \neg y$ n'en est pas une)
- Formule CNF : conjonction de clauses (au moins 1, ou plusieurs clauses)
- Une interprétation satisfait une clause si elle satisfait au moins un littéral dans la clause (car c'est une disjonction)
- Une interprétation satisfait une CNF si elle satisfait toutes les clauses (car c'est une conjonction)
- Un solveur SAT est un algorithme qui prend en entrée une CNF et qui détermine si elle est SATisfiable, c'est-à-dire si elle a au moins un modèle
  - Le solveur SAT permet aussi de retourner un modèle, si il en existe

Transformation en CNF

- Élimination des doubles négations : $\neg\neg\phi \equiv \phi$
- Lois de De Morgan :
  - $\neg(\phi_1 \vee \phi_2) = \neg\phi_1 \wedge \neg\phi_2$
  - $\neg(\phi_1 \wedge \phi_2) = \neg\phi_1 \vee \neg\phi_2$
- Distributivité
  - $\phi_1 \vee (\phi_2 \wedge \phi_3) \equiv (\phi_1 \vee \phi_2) \wedge (\phi_1 \vee \phi_3)$
  - $\phi_1 \wedge (\phi_2 \vee \phi_3) \equiv (\phi_1 \wedge \phi_2) \vee (\phi_1 \wedge \phi_3)$

Transformation en CNF

- Élimination des doubles négations : $\neg\neg\phi \equiv \phi$
- Lois de De Morgan :
  - $\neg(\phi_1 \vee \phi_2) = \neg\phi_1 \wedge \neg\phi_2$
  - $\neg(\phi_1 \wedge \phi_2) = \neg\phi_1 \vee \neg\phi_2$
- Distributivité
  - $\phi_1 \vee (\phi_2 \wedge \phi_3) \equiv (\phi_1 \vee \phi_2) \wedge (\phi_1 \vee \phi_3)$
  - $\phi_1 \wedge (\phi_2 \vee \phi_3) \equiv (\phi_1 \wedge \phi_2) \vee (\phi_1 \wedge \phi_3)$
- L'application successive de ces règles permet de transformer n'importe quelle formule en une CNF équivalente
  - Attention, cette transformation peut mener à une formule exponentiellement plus grande que la formule d'origine
  - Il existe une méthode un peu plus complexe qui permet de garantir une taille linéaire (transformation de Tseytin)

Transformation en CNF

- Élimination des doubles négations : $\neg\neg\phi \equiv \phi$
- Lois de De Morgan :
  - $\neg(\phi_1 \vee \phi_2) = \neg\phi_1 \wedge \neg\phi_2$
  - $\neg(\phi_1 \wedge \phi_2) = \neg\phi_1 \vee \neg\phi_2$
- Distributivité
  - $\phi_1 \vee (\phi_2 \wedge \phi_3) \equiv (\phi_1 \vee \phi_2) \wedge (\phi_1 \vee \phi_3)$
  - $\phi_1 \wedge (\phi_2 \vee \phi_3) \equiv (\phi_1 \wedge \phi_2) \vee (\phi_1 \wedge \phi_3)$
- L'application successive de ces règles permet de transformer n'importe quelle formule en une CNF équivalente
  - Attention, cette transformation peut mener à une formule exponentiellement plus grande que la formule d'origine
  - Il existe une méthode un peu plus complexe qui permet de garantir une taille linéaire (transformation de Tseytin)

Vous pouvez prouver que ces règles fonctionnent en utilisant la méthode des tables de vérité

Fonctionnement des solveurs SAT

- La plupart des solveurs SAT utilisant l'algorithme DPLL (Davis–Putnam–Logemann–Loveland)
- Principe de base :

Fonctionnement des solveurs SAT

- La plupart des solveurs SAT utilisant l'algorithme DPLL (Davis–Putnam–Logemann–Loveland)
- Principe de base :
  - Choisir une variable x_i et lui assigner une valeur de vérité (par exemple 1)

Fonctionnement des solveurs SAT

- La plupart des solveurs SAT utilisant l'algorithme DPLL (Davis–Putnam–Logemann–Loveland)
- Principe de base :
  - Choisir une variable x_i et lui assigner une valeur de vérité (par exemple 1)
  - Toutes les clauses qui contiennent x_i sont satisfaites, elles peuvent être supprimées

Fonctionnement des solveurs SAT

- La plupart des solveurs SAT utilisant l'algorithme DPLL (Davis–Putnam–Logemann–Loveland)
- Principe de base :
  - Choisir une variable x_i et lui assigner une valeur de vérité (par exemple 1)
  - Toutes les clauses qui contiennent x_i sont satisfaites, elles peuvent être supprimées
  - Toutes les clauses qui contiennent $\neg x_i$ peuvent être raccourcies : $x_1 \vee x_2 \vee \neg x_i$ devient $x_1 \vee x_2$

Fonctionnement des solveurs SAT

- La plupart des solveurs SAT utilisant l'algorithme DPLL (Davis–Putnam–Logemann–Loveland)
- Principe de base :
  - Choisir une variable x_i et lui assigner une valeur de vérité (par exemple 1)
  - Toutes les clauses qui contiennent x_i sont satisfaites, elles peuvent être supprimées
  - Toutes les clauses qui contiennent $\neg x_i$ peuvent être raccourcies : $x_1 \vee x_2 \vee \neg x_i$ devient $x_1 \vee x_2$
  - Si on obtient une clause $\ll$ vide $\gg$ (i.e. tous les littéraux sont falsifiés), c'est qu'on a pris une décision incohérente : on revient en arrière et on essaye d'assigner la valeur 0 à x_1

Fonctionnement des solveurs SAT

- La plupart des solveurs SAT utilisant l'algorithme DPLL (Davis–Putnam–Logemann–Loveland)
- Principe de base :
  - Choisir une variable x_i et lui assigner une valeur de vérité (par exemple 1)
  - Toutes les clauses qui contiennent x_i sont satisfaites, elles peuvent être supprimées
  - Toutes les clauses qui contiennent $\neg x_i$ peuvent être raccourcies : $x_1 \vee x_2 \vee \neg x_i$ devient $x_1 \vee x_2$
  - Si on obtient une clause $\ll$ vide $\gg$ (i.e. tous les littéraux sont falsifiés), c'est qu'on a pris une décision incohérente : on revient en arrière et on essaye d'assigner la valeur 0 à x_1
  - Si 0 mène aussi à une incohérence, alors la formule n'est pas satisfiable (aucun modèle)

Fonctionnement des solveurs SAT

- La plupart des solveurs SAT utilisant l'algorithme DPLL (Davis–Putnam–Logemann–Loveland)
- Principe de base :
  - Choisir une variable x_i et lui assigner une valeur de vérité (par exemple 1)
  - Toutes les clauses qui contiennent x_i sont satisfaites, elles peuvent être supprimées
  - Toutes les clauses qui contiennent $\neg x_i$ peuvent être raccourcies : $x_1 \vee x_2 \vee \neg x_i$ devient $x_1 \vee x_2$
  - Si on obtient une clause $\ll$ vide $\gg$ (i.e. tous les littéraux sont falsifiés), c'est qu'on a pris une décision incohérente : on revient en arrière et on essaye d'assigner la valeur 0 à x_1
  - Si 0 mène aussi à une incohérence, alors la formule n'est pas satisfiable (aucun modèle)
  - Sinon, on continue jusqu'à avoir trouvé une valeur pour chaque variable, ce qui donne un modèle

Fonctionnement des solveurs SAT

- La plupart des solveurs SAT utilisant l'algorithme DPLL (Davis–Putnam–Logemann–Loveland)
- Principe de base :
  - Choisir une variable x_i et lui assigner une valeur de vérité (par exemple 1)
  - Toutes les clauses qui contiennent x_i sont satisfaites, elles peuvent être supprimées
  - Toutes les clauses qui contiennent $\neg x_i$ peuvent être raccourcies : $x_1 \vee x_2 \vee \neg x_i$ devient $x_1 \vee x_2$
  - Si on obtient une clause $\ll$ vide $\gg$ (i.e. tous les littéraux sont falsifiés), c'est qu'on a pris une décision incohérente : on revient en arrière et on essaye d'assigner la valeur 0 à x_1
  - Si 0 mène aussi à une incohérence, alors la formule n'est pas satisfiable (aucun modèle)
  - Sinon, on continue jusqu'à avoir trouvé une valeur pour chaque variable, ce qui donne un modèle
- Il existe des techniques pour simplifier la formule
  - Propagation unitaire : s'il y a une clause unitaire (e.g. x ou $\neg y$), on peut propager dans toutes les clauses (x vaut 1 et y vaut 0 partout)

Fonctionnement des solveurs SAT

- La plupart des solveurs SAT utilisant l'algorithme DPLL (Davis–Putnam–Logemann–Loveland)
- Principe de base :
  - Choisir une variable x_i et lui assigner une valeur de vérité (par exemple 1)
  - Toutes les clauses qui contiennent x_i sont satisfaites, elles peuvent être supprimées
  - Toutes les clauses qui contiennent $\neg x_i$ peuvent être raccourcies : $x_1 \vee x_2 \vee \neg x_i$ devient $x_1 \vee x_2$
  - Si on obtient une clause $\ll$ vide $\gg$ (i.e. tous les littéraux sont falsifiés), c'est qu'on a pris une décision incohérente : on revient en arrière et on essaye d'assigner la valeur 0 à x_1
  - Si 0 mène aussi à une incohérence, alors la formule n'est pas satisfiable (aucun modèle)
  - Sinon, on continue jusqu'à avoir trouvé une valeur pour chaque variable, ce qui donne un modèle
- Il existe des techniques pour simplifier la formule
  - Propagation unitaire : s'il y a une clause unitaire (e.g. x ou $\neg y$), on peut propager dans toutes les clauses (x vaut 1 et y vaut 0 partout)
  - Littéraux purs : si une variable apparaît avec la même polarité dans toutes les clauses (par exemple, x apparaît dans certaines clauses mais $\neg x$ n'apparaît nulle part) on peut aussi propager de l'information (on assigne à x la valeur 1 et les clauses correspondantes sont satisfaites)

DPLL et recherche arborescente

DPLL partage des points communs avec la recherche arborescente

- États : ensembles de variables assignées ou non à une valeur de vérité
- Actions : Assigner une valeur 0 ou 1 à une variable
- États buts : toutes les variables ont une valeur et toutes les clauses sont satisfaites
- Il existe aussi des heuristiques (e.g. pour choisir en priorité des variables qui permettront de trouver rapidement une solution)

DPLL et recherche arborescente

DPLL partage des points communs avec la recherche arborescente

- États : ensembles de variables assignées ou non à une valeur de vérité
- Actions : Assigner une valeur 0 ou 1 à une variable
- États buts : toutes les variables ont une valeur et toutes les clauses sont satisfaites
- Il existe aussi des heuristiques (e.g. pour choisir en priorité des variables qui permettront de trouver rapidement une solution)
- Le fait d'utiliser des CNF et des techniques comme la propagation unitaire, l'élimination des littéraux purs (et d'autres techniques plus complexes) permet de résoudre le problème beaucoup plus efficacement qu'avec une recherche arborescente ≪ simple ≫

Conclusion sur SAT

- La logique propositionnelle et les solveurs SAT sont des outils très puissants pour modéliser des problèmes complexes et les résoudre « simplement »
- Aujourd'hui, les solveurs SAT peuvent traiter (assez rapidement) des modèles avec des milliers (voire millions) de variables et de clauses
- Pour la partie pratique associée à ce cours, vous utiliserez une API Python qui permet de modéliser un problème sous forme de clauses et de le résoudre directement (sans avoir besoin d'écrire soi-même l'algorithme DPLL !)

Programmation par Contraintes

Principe de base

- Intuition :
  - ensemble de variables qui décrivent un problème
  - ensemble de contraintes (logiques, arithmétiques, etc) sur ces variables
- SAT est une forme de programmation par contraintes avec des restrictions (les variables sont booléennes, et les contraintes des clauses)
- Problème de satisfaction de contraintes (CSP) : représentation de domaines et contraintes plus riches qu'avec SAT
  - domaines : nombres, lettres, couleurs, objets quelconques,...
  - contraintes : de manière générale, n'importe quelle combinaison de valeurs possibles pour les variables. Souvent représentées e.g. par des contraintes arithmétiques ($x \geq y$, $x \neq y$, $x + y \geq z + w$, ...)

Principe de base

- Intuition :
  - ensemble de variables qui décrivent un problème
  - ensemble de contraintes (logiques, arithmétiques, etc) sur ces variables
- SAT est une forme de programmation par contraintes avec des restrictions (les variables sont booléennes, et les contraintes des clauses)
- Problème de satisfaction de contraintes (CSP) : représentation de domaines et contraintes plus riches qu'avec SAT
  - domaines : nombres, lettres, couleurs, objets quelconques,...
  - contraintes : de manière générale, n'importe quelle combinaison de valeurs possibles pour les variables. Souvent représentées e.g. par des contraintes arithmétiques ($x \geq y$, $x \neq y$, $x + y \geq z + w$, ...)
- L'utilisation de domaines de variables et de contraintes plus riches qu'avec SAT peut permettre une modélisation plus facile du problème

Exemple classique : coloriage de cartes

- Règles : on doit colorier une carte avec k couleurs, sans que deux pays voisins soient de la même couleur
- Selon la carte et le nombre de couleurs k, il n'y a pas forcément de solution
- Variables : $\{WA, NT, Q, SA, NSW, V, T\}$, avec le même domaine e.g. $\{R, V, B\}$
- Contraintes : $WA \neq NT$, $WA \neq SA$, $NT \neq SA$, $NT \neq Q$, $SA \neq Q$, $SA \neq NSW$, $SA \neq V$, $Q \neq NSW$, $NSW \neq V$
- Solution : affectation de couleurs qui respecte toutes les contraintes

Pourquoi pas la recherche arborescente ?

- Tout comme pour l'algorithme DPLL des solveurs SAT, les solveurs CSP utilisent une méthode similaire à la recherche arborescente
  - États : combinaisons de couples (variable, valeur)
  - Actions : assigner à une variable une valeur de son domaine
  - États buts : toutes les variables ont une valeur et toutes les contraintes sont satisfaites

Pourquoi pas la recherche arborescente ?

- Tout comme pour l'algorithme DPLL des solveurs SAT, les solveurs CSP utilisent une méthode similaire à la recherche arborescente
  - États : combinaisons de couples (variable, valeur)
  - Actions : assigner à une variable une valeur de son domaine
  - États buts : toutes les variables ont une valeur et toutes les contraintes sont satisfaites
- Et comme pour SAT, des techniques permettent d'accélérer la recherche
  - E.g. quand on assigne une valeur à une variable, ça a un impact sur d'autres variables qui sont reliées par des contraintes (e.g. si $WA = R$, on retire cette valeur du domaine de NT et SA) : possibilité de réduire l'espace de recherche et de détecter des conflits

Pourquoi pas SAT ?

- On peut traduire un CSP en problème SAT, par exemple :
  - *wa_bleu*, *wa_rouge*, *wa_vert*, etc : variables booléennes pour indiquer « Western Australia est en bleu/rouge/vert »
  - Contraintes logiques : $wa_bleu \vee wa_rouge \vee wa_vert$, $\neg(wa_bleu \wedge wa_rouge)$, $\neg(wa_bleu \wedge wa_vert)$, $\neg(wa_rouge \wedge wa_vert)$, etc
- En général, la modélisation CSP est plus compacte et plus intuitive (on verra des contraintes très riches, difficiles à exprimer en CNF)
- Remarque : quand un encodage SAT est possible, un solveur SAT sera (en général) plus rapide que le solveur CSP
  $\Rightarrow$ compromis entre expressivité pour la modélisation et rapidité de la résolution

Contraintes Globales : All Different

- Contraintes avec un sens précis qui reviennent souvent
- Exemple classique : AllDiff (pour *all different*)
  - Cette contrainte globale porter une un ensemble de variables quelconque, et exprime l'obligation de donner une valeur différente à chaque variable
  - Utile pour des problèmes comme le Sudoku : $x_{i,j}$ représente la valeur de la case sur la ligne i et colonne j (dans $\{1, \ldots, 9\}$)
  - $\texttt{allDiff}(x_{1,1}, x_{1,2}, \ldots, x_{1,9})$: toutes les cases de la première ligne doivent contenir une valeur différente
  - On reproduit cette contrainte pour les autres lignes, les colonnes, et les secteurs 3×3
- Beaucoup plus dur à représenter en SAT !

$x_{1,1}$	$x_{1,2}$	$x_{1,3}$	$x_{1,4}$	$x_{1,5}$	$x_{1,6}$	$x_{1,7}$	$x_{1,8}$	$x_{1,9}$	$\texttt{allDiff}(x_{1,1}, x_{1,2}, \ldots, x_{1,9})$
8			2		3			6	$\texttt{allDiff}(x_{2,1}, x_{2,2}, \ldots, x_{2,9})$
	3			6			7		
		1				6			$\vdots$
5	4						1	9	
		2				7			
	9			3			8		
2			8		4			7	
	1		9		7		6		$\texttt{allDiff}(x_{9,1}, x_{9,2}, \ldots, x_{9,9})$

Contraintes globales : quelques exemples

- La plupart des contraintes globales sont associées à des mécanismes de propagation dédiés : l'arbre de recherche peut être « coupé » pour accélérer la recherche de solutions
- `atLeastNValues`$(n, x_1, \ldots, x_k)$: les variables x_1, ..., x_k doivent avoir au moins n valeurs différentes
- `allEqual`$(x_1, \ldots, x_k)$: l'inverse du `allDiff`
- `disjoint`$(x_1, \ldots, x_k, y_1, \ldots, y_l)$: chaque variable x_i doit avoir une valeur distincte de toutes les valeurs des variables y_i
- `increasingSum`$(x_1, \ldots, x_k, y)$: les variables x_1, ..., x_k ont des valeurs rangées dans l'ordre croissant, et leur somme est égale à y
- Etc
- Un catalogue des (423) contraintes étudiées dans la littérature est disponible : `https://sofdem.github.io/gccat/`

Optimisation

- Dans certains cas, toutes les solutions n'ont pas la même « valeur »
  - Coût à minimiser
  - Utilité à maximiser
- Exemple :
  - Un parent partage 30 euros d'argent de poche entre ses enfants, A (14 ans), B (12 ans) et C (10 ans)
  - Il n'y a que des billets de 5 euros, le parent donne un peu plus d'argent en fonction de l'âge, et il ne dépasse pas 20 euros par enfant
  - On peut exprimer cela comme un CSP avec trois variables A, B, C dont le domaine est $\{0, 5, \ldots, 20\}$ et les contraintes $A \geq B + 5$, $B \geq C + 5$, $A + B + C = 30$
  - Deux solutions : $A = 20, B = 10, C = 0$ et $A = 15, B = 10, C = 5$
  - Pour être plus équitable, le parent peut souhaiter maximiser la valeur de C (pour limiter la jalousie entre les enfants), dans ce cas la deuxième solution est meilleure

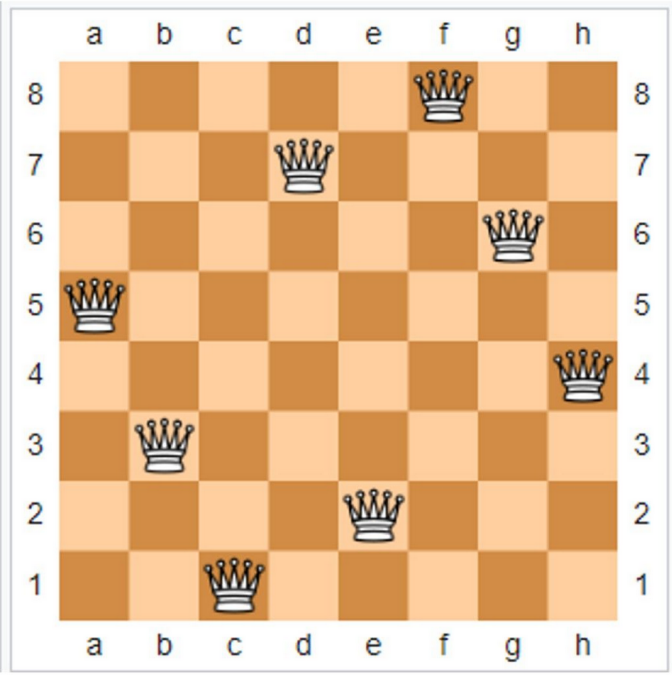
Optimisation

- Dans certains cas, toutes les solutions n'ont pas la même « valeur »
  - Coût à minimiser
  - Utilité à maximiser
- Exemple :
  - Un parent partage 30 euros d'argent de poche entre ses enfants, A (14 ans), B (12 ans) et C (10 ans)
  - Il n'y a que des billets de 5 euros, le parent donne un peu plus d'argent en fonction de l'âge, et il ne dépasse pas 20 euros par enfant
  - On peut exprimer cela comme un CSP avec trois variables A, B, C dont le domaine est $\{0, 5, \ldots, 20\}$ et les contraintes $A \geq B + 5$, $B \geq C + 5$, $A + B + C = 30$
  - Deux solutions : $A = 20, B = 10, C = 0$ et $A = 15, B = 10, C = 5$
  - Pour être plus équitable, le parent peut souhaiter maximiser la valeur de C (pour limiter la jalousie entre les enfants), dans ce cas la deuxième solution est meilleure
- Formellement, un problème d'optimisation de contraintes (COP) est un CSP auquel on ajoute une fonction objectif, c'est-à-dire une fonction qui dépend des variables du CSP et dont on doit maximiser (ou minimiser) la valeur

Conclusion

- SAT et CSP sont dans le domaine de la programmation déclarative : on décrit le problème à résoudre (sous forme de clauses/contraintes) et un outil existant le résoud pour nous !
- Étape clé : la modélisation
  - choix des variables (booléennes ou dans des domaines discrets)
  - définition des contraintes
- Les algorithmes pour SAT et CSP peuvent être vus comme des formes améliorées de recherche arborescente, qui permettent dans certains cas de gagner du temps de façon exponentielle
- Remarque : il existe beaucoup de formes plus complexes (plusieurs formes d'optimisation, utilisation de domaines continus pour les variables, etc)

Exercice : le problème des n reines



- On doit disposer n reines sur un échiquier de taille $n \times n$, en s'assurant qu'elles ne sont pas en mesure de se menacer les unes les autres
- Rappel : aux échecs, une reine peut se déplacer (et donc menacer d'autres pièces) sur n'importe quelle distance, en ligne, en colonne ou en diagonale
- On considère le cas $n = 8$. Essayez de modéliser ce problème en SAT et en CSP